

Classification in Machine Learning Notes

Classification in Machine Learning

Classification is a **supervised learning** technique used to categorize data into predefined classes or labels. Given a dataset with input features and corresponding labels, a classification algorithm learns patterns and relationships to predict the class of new, unseen data points.

In supervised learning, classification refers to predicting a categorical (discrete) label

$$y \in \{C_1, C_2, \dots, C_k\}$$

given input features x .

Unlike regression, where y is continuous, classification's goal is to decide the class membership. Binary classification corresponds to $k = 2$, while multiclass classification corresponds to $k > 2$.

Common classification algorithms include:

- **Logistic Regression** — suitable for binary classification, or extended to multiclass via one-vs-rest.
- **k-Nearest Neighbors (kNN)** — classifies based on the majority class among the k nearest data points.
- **Decision Trees** — hierarchical models that split data based on feature thresholds.
- **Random Forests** — ensembles of decision trees for improved accuracy and robustness.
- **Support Vector Machines (SVM)** — find optimal hyperplanes that separate classes in feature space.
- **Naïve Bayes** — probabilistic model based on Bayes' theorem assuming feature independence.

```
# Import libraries
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from matplotlib.colors import ListedColormap

# Helper function to plot decision boundaries
def plot_decision_boundary(model, X, y, ax, title):
    h = .02 # step size in the mesh
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                          np.arange(y_min, y_max, h))
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
```

```

# Define color map
colors = ('#c7bce5', '#ffffb3', '#b3e2cd', '#fbb4ae', '#decbe4')
cmap = ListedColormap(colors[:len(np.unique(y))])

# Plot contour
ax.contourf(xx, yy, Z, alpha=0.4, cmap=cmap)
scatter = ax.scatter(X[:, 0], X[:, 1], c=y, s=30, edgecolor='k', cmap=cmap)
ax.set_title(title)
ax.set_xlabel('Feature 1')
ax.set_ylabel('Feature 2')
ax.legend(*scatter.legend_elements(), title="Classes")

# ---- First: Simple Binary Classification ----
X_binary, y_binary = make_classification(
    n_samples=100, n_features=2, n_redundant=0,
    n_informative=2, n_clusters_per_class=1, random_state=2
)

model_binary = LogisticRegression()
model_binary.fit(X_binary, y_binary)

# ---- Second: Multi-Class Classification ----
X_multi, y_multi = make_classification(
    n_samples=200, n_features=2, n_redundant=0, n_informative=2,
    n_clusters_per_class=1, n_classes=4, random_state=42
)

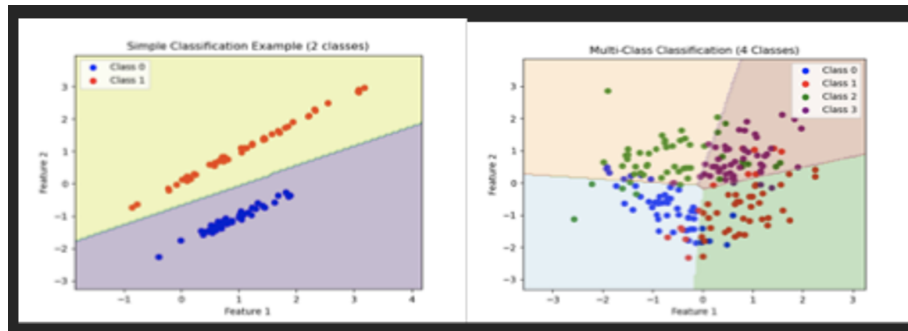
model_multi = LogisticRegression(multi_class='ovr')
model_multi.fit(X_multi, y_multi)

# ---- Plot ----
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
plot_decision_boundary(model_binary, X_binary, y_binary, axes[0],
    "Simple Classification Example (2 classes)")
plot_decision_boundary(model_multi, X_multi, y_multi, axes[1],
    "Multi-Class Classification (4 classes)")
plt.tight_layout()
plt.show()

```

Applications of Classification

- **Spam Detection:** Classifying emails as spam or not spam.
- **Medical Diagnosis:** Predicting diseases based on symptoms.



- **Sentiment Analysis:** Categorizing text as positive, negative, or neutral.
- **Fraud Detection:** Identifying fraudulent transactions.
- **Image Recognition:** Classifying objects in images.

Types of Classification

Classification problems can be categorized into:

- **Binary Classification:** The target variable has only two possible classes (e.g., “yes” or “no”). Example: Predicting whether an email is spam (1) or not spam (0).
- **Multiclass Classification:** The target variable has more than two classes. Example: Hand-written digit recognition (digits 0–9).
- **Multilabel Classification:** Each instance can belong to multiple classes simultaneously. Example: Tagging a news article with multiple categories (“sports”, “politics”, etc.).
- **Imbalanced Classification:** One class is significantly underrepresented compared to others. Example: Fraud detection, where fraudulent transactions are rare.

Classification Algorithms

Several classification algorithms exist, each with unique characteristics:

1. Logistic Regression

A statistical model that applies the sigmoid function to predict probabilities for binary classification.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
```

```

# Load dataset
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Train model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Predict and evaluate
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

```

2. Decision Trees

Tree-like structures where decisions are made by splitting data based on feature values.

```

from sklearn.tree import DecisionTreeClassifier

# Train model
clf = DecisionTreeClassifier()
clf.fit(X_train, y_train)

# Predict and evaluate
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

```

3. Random Forest

An ensemble of multiple decision trees to improve accuracy and reduce overfitting.

```

from sklearn.ensemble import RandomForestClassifier

# Train model
clf = RandomForestClassifier(n_estimators=100)
clf.fit(X_train, y_train)

# Predict and evaluate
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))

```

4. Support Vector Machines (SVM)

Finds an optimal hyperplane that maximizes the margin between different classes.

```

from sklearn.svm import SVC

```

```
# Train model
clf = SVC(kernel='linear')
clf.fit(X_train, y_train)

# Predict and evaluate
y_pred = clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Model Evaluation

To assess the performance of a classification model and ensure it generalizes well to unseen data, the model needs to be evaluated statistically. The goal is to measure the accuracy and reliability of the model, identify overfitting or underfitting, and compare the performance of different models.

Model Evaluation for Classification Models

A **confusion matrix** is a table that summarizes the performance of a classification model by displaying the counts of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN).

Example Scenario:

- 100 patients
- **True Figures from the Hospital:**
 - 70 actually had COVID
 - 30 did not have COVID
- **Figures from the Model:**
 - 60 patients predicted to have COVID
 - 40 patients predicted not to have COVID

Confusion Matrix Terms

- **True Positives (TP):** Correctly predicted positive classes.
- **True Negatives (TN):** Correctly predicted negative classes.
- **False Positives (FP):** Incorrectly predicted positive classes (Type I error).
- **False Negatives (FN):** Incorrectly predicted negative classes (Type II error).

Performance Metrics

- **Accuracy:** Measures the proportion of correctly predicted instances.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Note: Accuracy can be misleading in imbalanced datasets.

- **Precision:** Measures the proportion of correctly predicted positive instances out of all predicted positive instances. Useful when the cost of FP is high.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall / Sensitivity / True Positive Rate (TPR):** Measures the proportion of correctly predicted positive instances out of all actual positive instances. Useful when the cost of FN is high.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Harmonic mean of Precision and Recall. Useful when seeking a balance between precision and recall.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **False Positive Rate (FPR):** Measures the proportion of actual negative instances incorrectly classified as positive. It quantifies how many false alarms a classification model generates.

$$\text{FPR} = \frac{FP}{FP + TN}$$

- **ROC Curve:** Plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings.
- **AUC (Area Under the Curve):** Measures the entire area under the ROC curve.

$$\text{AUC} = 1 \Rightarrow \text{Perfect Classifier}, \quad \text{AUC} = 0.5 \Rightarrow \text{Random Classifier}$$